PROJECT DOCUMENTATION

EduGenie

Google Gemini Powered Learning Assistant

Ask • Explain • Summarize • Quiz • Learning Path

Built with FastAPI | HTML, CSS and JavaScript | Google Gemini API

Version 1.0

Contents

1 Introduction and Objective
2 Key Features
3 Technology Stack
4 System Architecture
5 Project Structure
6 Module Details with Code
7 REST API Reference
8 Web Portal Output
9 Setup and Run Locally
10 Deployment on Render
11 Demo Guide
12 Troubleshooting
13 Future Enhancements
14 Conclusion

1. Introduction and Objective

EduGenie is a lightweight, AI-powered educational assistant. It helps learners ask questions, understand difficult concepts, summarize study material, test their knowledge and follow a structured learning path, all from one calm and simple web page.

The application uses the **Google Gemini API** for content generation and a **FastAPI** backend to connect the browser to five focused learning modules.

Project objective

> Build an educational assistant that helps learners ask questions, understand difficult concepts, summarize study material, test their knowledge, and follow structured learning paths.

Why EduGenie

- **One place for five study tasks**, so students do not switch between tools.
- **Beginner-friendly language** in every response.
- **Automatic model fallback**, so the app keeps working if one Gemini model is unavailable.
- **Simple to deploy**, with a ready Render blueprint and a health-check endpoint.

2. Key Features

Feature	What it does	Endpoint
Smart Q&A	Gives a direct, student-friendly answer to any educational question.	POST /qa
Simple Explanations	Explains a topic in 5 fixed sections: definition, how it works, example, key points, recap.	POST /explain
Instant Summaries	Turns long study text into short revision notes with a one-line takeaway.	POST /summarize
Interactive Quizzes	Creates exactly 3 MCQs with 4 options each and shows instant feedback and a score.	POST /quiz
Learning Paths	Builds a roadmap from prerequisites to a final project, with weekly schedule.	POST /learn/recommendations

Other highlights

- Responsive, presentation-ready interface.
- Interactive FastAPI documentation at **/docs**.
- Health check at **/health** for deployment platforms.
- Secrets are kept in environment variables, never in source code.

3. Technology Stack

Layer	Technology	Purpose
Frontend	HTML5, CSS3, JavaScript	Page structure, responsive design, API calls and quiz interaction
Backend	Python, FastAPI, Uvicorn	REST API and ASGI server
Templates	Jinja2	Serves the main page (index.html)
AI	Google Gemini API (google-genai SDK), Gemini 3.8 Flash	Generates answers, explanations, summaries, quizzes and paths
Validation	Pydantic	Request model and quiz response schema
Config	python-dotenv	Loads GEMINI_API_KEY and GEMINI_MODEL from .env
Hosting	Render	Cloud deployment via render.yaml

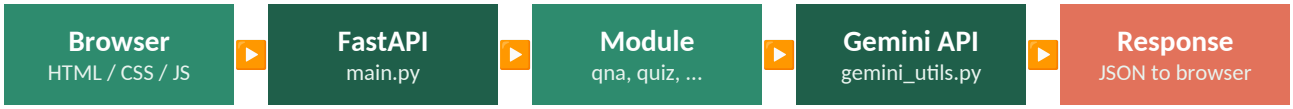
Python dependencies (requirements.txt)

requirements.txt

```
fastapi
uvicorn[standard]
jinja2
python-dotenv
google-genai
pydantic
```

4. System Architecture

EduGenie follows a simple request and response pipeline. Each learning tool has its own small Python module, which keeps the code easy to read and extend.



How a request flows

1. The user picks a tool (Ask, Explain, Summarize, Quiz or Learn Path) and enters text.
2. JavaScript sends a **POST** request to the matching FastAPI endpoint.
3. FastAPI validates the body with Pydantic (**TextRequest**) and checks that Gemini is configured.
4. The endpoint calls the matching Python module, which builds an educational prompt.
5. **generate_with_fallback()** sends the prompt to Gemini, trying backup models if needed.
6. The result is returned as JSON, and the frontend renders it in the response panel.

5. Project Structure

EduGenie/

```
EduGenie/
|-- main.py                 FastAPI app and endpoints
|-- gemini_utils.py         Model list and fallback logic
|-- qna.py                  Question answering
|-- explanation_module.py   Beginner explanations
|-- summary_module.py       Revision summaries
|-- quiz_module.py          3-question MCQ quiz (structured output)
|-- learning_path.py        Learning roadmap
|-- requirements.txt        Python dependencies
|-- render.yaml             Render deployment blueprint
|-- .env.example            Environment variable template
|-- .gitignore
|-- README.md
|-- templates/
|   `-- index.html          Main web page
`-- static/
    |-- style.css           Responsive styling
    `-- app.js              Frontend logic and quiz interaction
```

Module mapping

File	Responsibility
main.py	Creates the FastAPI app, CORS, static files, templates and all REST endpoints.
gemini_utils.py	Normalizes model names and retries with fallback Gemini models.
qna.py	General educational question answering.
explanation_module.py	Turns difficult topics into beginner-friendly explanations.
summary_module.py	Creates concise, revision-oriented summaries.
quiz_module.py	Generates exactly three MCQs with four options each, validated by Pydantic.
learning_path.py	Generates a beginner-to-advanced learning roadmap.

6. Module Details with Code

This section shows the most important code from the project.

6.1 Application setup (main.py)

The app loads the API key from **.env**, creates the Gemini client only when a key exists, and defaults to the model **gemini-3.8-flash**.

main.py: configuration

```
load_dotenv(BASE_DIR / ".env")

API_KEY = os.getenv("GEMINI_API_KEY")
MODEL_NAME = os.getenv("GEMINI_MODEL") or "gemini-3.8-flash"
client = genai.Client(api_key=API_KEY) if API_KEY else None

app = FastAPI(
    title="EduGenie",
    description="Google Gemini Powered Learning Assistant",
    version="1.0.0",
)
```

6.2 Endpoint pattern (main.py)

Every endpoint follows the same three steps: validate the input, make sure Gemini is configured, then call the module and return JSON.

main.py: /qa endpoint and helpers

```
class TextRequest(BaseModel):
    text: str

@app.post("/qa")
async def qa(request: TextRequest):
    validate_text(request.text, "question")
    require_gemini_client()
    try:
        return {"success": True,
                "result": answer_question(client, request.text, MODEL_NAME)}
    except Exception as exc:
        raise HTTPException(status_code=500, detail=safe_error(exc))

def validate_text(value: str, field_name: str) -> None:
    if not value or not value.strip():
        raise HTTPException(status_code=400,
                            detail=f"Please enter a {field_name}.")
```

6.3 Automatic model fallback (gemini_utils.py)

If the chosen model is missing, rate-limited or temporarily unavailable, EduGenie tries the next model in the list instead of failing.

gemini_utils.py: fallback list and retry loop

```
DEFAULT_MODELS = [
    "gemini-3.8-flash",
    "gemini-3.5-flash-lite",
    "gemini-3.5-flash",
    "gemini-2.5-flash",
]

def generate_with_fallback(client, prompt, preferred_model=None,
                           *, response_schema=None):
    last_error = None
    for model_name in get_model_candidates(preferred_model):
        try:
            request_kwargs = {"model": model_name, "contents": prompt}
            if response_schema is not None:
                request_kwargs["config"] = {
                    "response_mime_type": "application/json",
                    "response_schema": response_schema,
                }
            return client.models.generate_content(**request_kwargs)
        except Exception as exc:
            last_error = exc
            if _should_try_next_model(exc):
                continue
            raise
    raise last_error
```

Retry rules: the next model is tried on HTTP 404, 429, 500, 502, 503 or 504, or when the error message mentions "not found", "unsupported" or "unavailable".

6.4 Smart Q&A (qna.py)

qna.py

```
def answer_question(client, question: str, model_name: str) -> str:
    prompt = f"""
You are EduGenie, an educational AI assistant.

Answer the student's question accurately and clearly.

Student question:
{question}

Instructions:
- Give the direct answer first.
- Use simple, student-friendly language.
- Explain difficult terms when necessary.
- Use short sections or bullets when helpful.
- Do not unnecessarily make the response long.
"""
    response = generate_with_fallback(client, prompt, model_name)
    return response.text or "No answer was generated."
```

6.5 Explanation, summary and learning path

Module	Prompt design
explanation_module.py	Tutor persona. Forces exactly 5 sections: Simple definition, How it works, Easy example, Important points, Quick recap.
summary_module.py	Preserves facts, removes repetition, uses bullets and ends with a one-sentence revision takeaway.
learning_path.py	7 sections (prerequisites to final project). Each level lists topics, outcomes and time. Recommends resource types only and never invents URLs.

6.6 Quiz with structured output (quiz_module.py)

The quiz uses a **Pydantic schema** so Gemini must return valid JSON with exactly 3 questions and 4 options each. The reply is validated before it is sent to the browser.

quiz_module.py: schema and generation

```
class QuizQuestion(BaseModel):
    question: str
    options: List[str] = Field(min_length=4, max_length=4)
    correct_answer: str

class QuizResponse(BaseModel):
    questions: List[QuizQuestion] = Field(min_length=3, max_length=3)

def generate_quiz(client, topic: str, model_name: str) -> dict:
    prompt = f"""Create a student-friendly multiple-choice quiz about:
{topic}
... Exactly 3 questions, exactly 4 options, one correct answer ..."""

    response = generate_with_fallback(
        client, prompt, model_name, response_schema=QuizResponse,
    )
    quiz = QuizResponse.model_validate_json(response.text)
    return quiz.model_dump()
```

7. REST API Reference

All POST endpoints accept the same JSON body and return **{ "success": true, "result": ... }**.

Method	Endpoint	Description	Input
GET	/	Main web interface	None
POST	/qa	Answer an educational question	text = question
POST	/explain	Beginner-friendly explanation	text = topic
POST	/summarize	Revision summary	text = study content
POST	/quiz	3-question MCQ quiz (JSON)	text = topic
POST	/learn/recommendations	Learning roadmap	text = topic
GET	/health	Service and Gemini status	None

Example request and response

```
POST /qa
{ "text": "Which is the largest ocean?" }

200 OK
{ "success": true, "result": "The Pacific Ocean is the largest ..." }
```

Error responses

Status	Meaning
400	The input text is empty ("Please enter a question / topic / text").
503	GEMINI_API_KEY is not set. Add it to .env and restart.
500	Gemini call failed. The message is trimmed to 500 characters by safe_error().

8. Web Portal Output

The portal opens at **http://127.0.0.1:8000**. The left card is where the learner chooses a tool and types input. The right card shows the AI response.

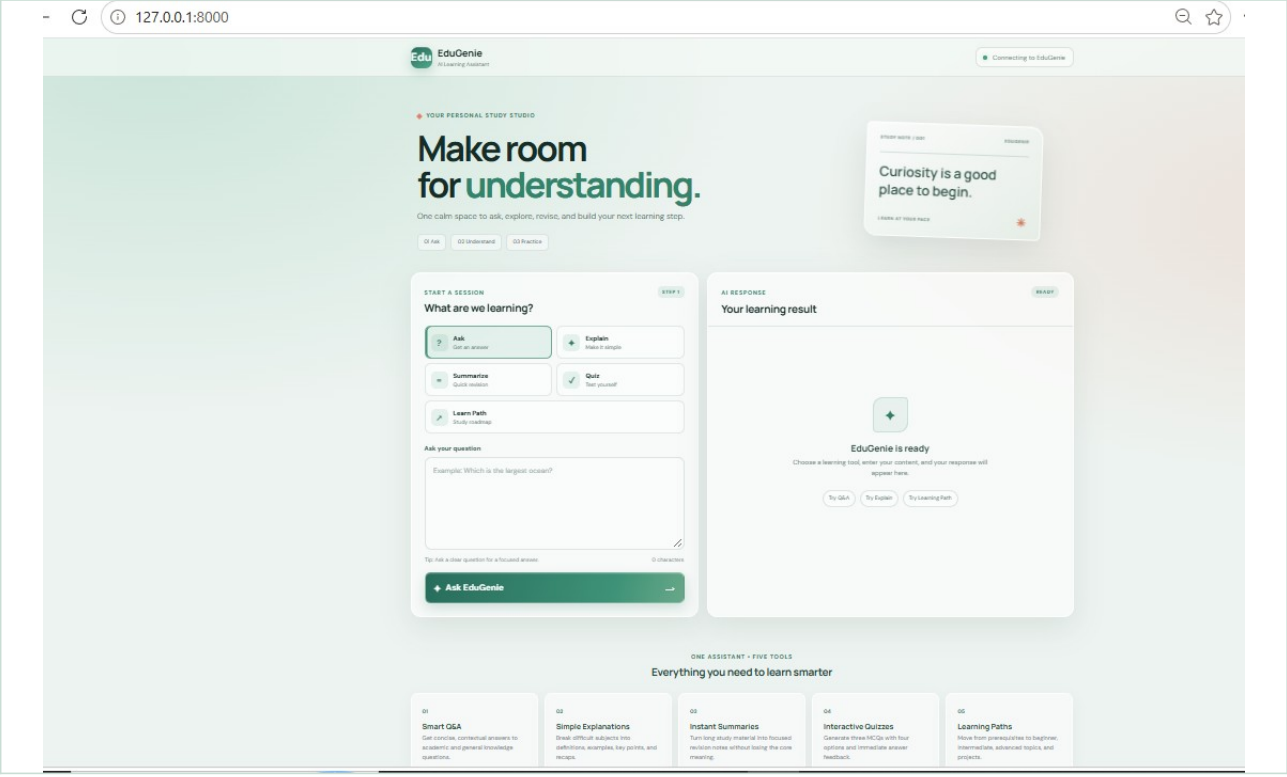


Figure 1. EduGenie home page running locally

Interface guide

Area	Description
Header	EduGenie logo and a status badge that shows the connection to the backend.
Hero section	Tagline "Make room for understanding" with three steps: Ask, Understand, Practice.
Tool selector	Five buttons: Ask, Explain, Summarize, Quiz and Learn Path.
Input box	Text area with a helpful placeholder, tip and character counter.
Action button	Sends the request to the selected endpoint (label changes with the tool).
AI Response panel	Shows the generated result. Quiz results appear as interactive questions with score.
Feature cards	Five cards below the fold that summarize each tool.

9. Setup and Run Locally

These steps are for **Windows** with **Python 3.10 or newer**.

7. Open Command Prompt in the project folder.
8. Create a virtual environment and activate it.
9. Install the dependencies.
10. Copy **.env.example** to **.env** and add your Gemini API key.
11. Start the server and open the browser.

Command Prompt

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env
uvicorn main:app --reload
```

.env

```
GEMINI_API_KEY=your_real_key_here
GEMINI_MODEL=gemini-3.8-flash
```

Page	URL
Web app	http://127.0.0.1:8000
FastAPI docs	http://127.0.0.1:8000/docs
Health check	http://127.0.0.1:8000/health

> **Security:** create your key in Google AI Studio. Never put the real API key in source code or GitHub, and do not commit the .env file.

10. Deployment on Render

The project includes a **render.yaml** blueprint, so deployment takes only a few clicks.

render.yaml

```
services:
  - type: web
    name: edugenie
    runtime: python
    buildCommand: pip install -r requirements.txt
    startCommand: uvicorn main:app --host 0.0.0.0 --port $PORT
    healthCheckPath: /health
    envVars:
      - key: GEMINI_API_KEY
        sync: false
      - key: GEMINI_MODEL
        value: gemini-3.8-flash
```

Option A: Blueprint

12. Push the project to GitHub.
13. In Render, create a **Blueprint** and select the repository.
14. Set **GEMINI_API_KEY** as a secret environment variable.
15. Deploy and open the generated Render URL.

Option B: Web Service

Setting	Value
Runtime	Python
Build command	pip install -r requirements.txt
Start command	uvicorn main:app --host 0.0.0.0 --port $PORT
Environment	GEMINI_API_KEY = your real key (optional: GEMINI_MODEL)

11. Demo Guide

Step	Tool	Sample input
1	Ask	Which is the largest ocean?
2	Explain	Pythagoras theorem
3	Summarize	Paste a paragraph about photosynthesis
4	Quiz	Photosynthesis (answer the 3 questions and show the score)
5	Learn Path	SQL
6	API docs	Open /docs to show the FastAPI endpoints

Viva explanation

> When a user selects a feature, JavaScript sends a POST request to the matching FastAPI endpoint. FastAPI validates the request using Pydantic. The endpoint calls the corresponding Python module. The module creates an educational prompt and sends it to Gemini. The Gemini response is returned as JSON to the browser, and the frontend renders the result.

12. Troubleshooting

Error: model not found (404)

During testing, the /qa endpoint returned this error:

Error output

```
500
{"detail":"404 NOT_FOUND ... models/gemini-1.5-flash is not found
for API version v1beta, or is not supported for generateContent ..."}
```

Cause: the requested model name is retired or not available for your API key.

Fix: set **GEMINI_MODEL** in .env (or on Render) to a model your key supports, then restart the server. EduGenie will also try the fallback models in gemini_utils.py.

Other common issues

Symptom	Likely cause	Solution
Status badge says "Connecting"	Backend not running or key missing	Start uvicorn, check /health, set GEMINI_API_KEY
503 error	API key not configured	Add GEMINI_API_KEY to .env and restart
400 error	Empty input	Type a question, topic or text first
429 error	Rate limit reached	Wait and retry. The fallback tries another model
Quiz fails to parse	Model returned invalid JSON	Retry, or choose a model that supports structured output

13. Future Enhancements

Area	Idea
Accounts and history	User authentication, learning history, persistent database
Progress	Progress dashboard and adaptive learning based on quiz performance
Input types	PDF and image input, voice interaction
Language	Multilingual learning
Institutions	Teacher and parent dashboard, LMS integration

14. Conclusion

EduGenie shows how a small FastAPI service and the Gemini API can deliver a complete study companion. Its modular design, structured quiz output, model fallback and one-click deployment make it easy to understand, demonstrate and extend.